Overview

This document is intended to serve as a companion to the final research poster for the Robotics & Drawing project. The following offers documentation/explainers regarding the implementation; the project's source code and usage information can be found here.

While abstraction is a keystone of all well-engineered projects, it is vital that all layers of a system be revealed for learning to occur. The following content will dive into the setup intricacies, technologies used, and design decisions involved with the creation of this project.

1. Hardware Foundation

What is the UR5 Robot (CB 3.0 Control Box)

The Universal Robots UR5 is a collaborative robotic arm (or "cobot") designed for precision tasks in industrial and research environments. Unlike traditional industrial robots that operate behind safety cages, collaborative robots like the UR5 are engineered to work safely alongside humans. The "UR5" designation refers to its 5-kilogram payload capacity, making it suitable for tasks requiring moderate lifting capability with high precision.

The UR5 features six rotational joints (called a "6-DOF" or six degrees-of-freedom configuration), allowing it to position and orient its end-effector in three-dimensional space with remarkable flexibility. Each joint contains precision encoders that provide real-time feedback about the robot's position, enabling precise control and safety monitoring.

Critical to this project is the CB 3.0 control box version. The control box serves as the robot's "brain," containing the computer that interprets commands, monitors safety systems, and controls the individual joint motors. The CB 3.0 (Control Box version 3.0) includes specific communication protocols and safety features that determine how external computers can interface with the robot. This version supports URCaps (Universal Robots Capability packages)—software plugins that extend the robot's functionality, including the External Control URCap essential for this project's operation.

PC Requirements and Setup

The computer controlling the robot must meet specific requirements to handle real-time communication and motion planning computations. The project utilized an AMD Ryzen 3950X processor, NVIDIA Quadro RTX 8000, and 32GB of RAM; providing sufficient computational power for simultaneous image processing, trajectory planning, and robot communication tasks.

More important than raw computational power is the operating system configuration. The PC runs Windows 10 as the host operating system, but the actual robot control software operates within a Linux virtual machine. This hybrid approach balances familiar Windows-based development tools with Linux's superior real-time performance and robotics software ecosystem.

Physical Network Connection

The robot connects to the PC through a direct Ethernet connection, bypassing complex network infrastructure that could introduce communication delays or reliability issues. This connection serves as the sole pathway for all command and status information between the computer and robot.

The UR5's control box features a standard Ethernet port that connects directly to the PC's network interface. While it's possible to route this connection through network switches or routers, direct connection eliminates potential points of failure and reduces communication latency—critical factors when controlling a robot that may be moving at speeds measured in meters per second.

This physical connection forms the foundation for all higher-level communication protocols. Every drawing command, safety signal, and status update travels through this single Ethernet cable, making its reliability paramount to the system's operation.

2. Software Environment Setup

Operating Systems: Windows vs Linux

An operating system serves as the fundamental software layer that manages a computer's hardware resources and provides a platform for applications to run. Think of it as the conductor of an orchestra, coordinating how different programs access the computer's memory, processor, and input/output devices.

Windows and Linux represent fundamentally different approaches to this coordination. Windows prioritizes user-friendly interfaces and broad compatibility with commercial software, making it the standard choice for design studios and creative workflows. Linux, conversely, emphasizes system transparency, customization, and precise control over system resources—qualities that prove essential for robotics applications.

The distinction becomes critical when controlling industrial robots. Linux's architecture allows for more predictable timing in how programs execute, while Windows' design prioritizes responsiveness to user interactions over consistent background task execution. This difference, seemingly minor in typical computer usage, becomes pronounced when sending time-sensitive commands to a robot arm moving through physical space.

Why Linux?

The fundamental reason for even needing to use Linux stems from the Robot Operating System (ROS)—the software framework that enables communication with the UR5 robot. Despite its name, ROS is not an operating system but rather a collection of software libraries and tools designed specifically for robotics applications.

ROS was developed primarily for Linux environments, with the vast majority of robotics hardware drivers, motion planning libraries, and control algorithms designed around Linux's architecture. While ROS2 (the newer version used in this project) technically supports Windows, the ecosystem remains heavily Linux-centric. Critical components like the UR5 robot drivers, MoveIt2 motion planning framework, and real-time control interfaces are optimized for Linux and often encounter compatibility issues or reduced functionality on Windows.

More importantly, the robotics community's standard practices, documentation, and troubleshooting resources assume Linux operation. Attempting to run ROS2 on Windows would mean navigating uncharted territory with limited community support and potentially unstable drivers—something that I didn’t want to deal with..

WSL2

In this project, the PC served as a shared resource in a studio environment, hosting Windows-specific design applications (such as Adobe Creative Suite, Rhino, and other architectural software) that could not be removed or replaced. This constraint ruled out converting the entire system to Linux, necessitating a virtualized approach.

The initial solution attempted was Windows Subsystem for Linux version 2 (WSL2). WSL2 creates a lightweight Linux environment within Windows, allowing Linux applications to run without full virtualization overhead. This seemed ideal—providing access to Linux tools while preserving the existing Windows applications.

However, WSL2 revealed a critical limitation during robot operation: the robot's movements appeared choppy and irregular, lacking the smooth, continuous motion expected from industrial robotic systems. This choppiness stemmed from WSL2's inability to support real-time kernel modifications, which are essential for consistent, predictable timing in robotics control systems.

Real-Time Kernel Requirements

A real-time kernel modifies how the operating system schedules tasks, prioritizing time-critical operations over general system responsiveness. In standard operating systems, the kernel may delay robot control commands to handle other system tasks like updating the display or managing background processes. For robotics applications, this unpredictable timing translates directly into jerky, inconsistent robot movements.

Real-time kernels guarantee that critical tasks (like sending robot position commands) execute within specified time windows, regardless of other system activity. This ensures smooth, predictable robot motion even when the computer is simultaneously running visualization software, processing images, or handling network traffic.

WSL2's architecture, designed for compatibility and ease-of-use rather than real-time performance, cannot accommodate the kernel modifications necessary for real-time operation. This fundamental limitation made WSL2 unsuitable for reliable robot control, despite its advantages in other contexts.

VirtualBox Linux Virtual Machine Solution

The final solution employed VirtualBox to create a complete Linux virtual machine within the Windows environment. Unlike WSL2's hybrid approach, VirtualBox creates a fully isolated Linux environment with its own kernel, allowing installation of real-time kernel patches and robotics-specific optimizations.

This approach successfully resolved the timing issues that plagued the WSL2 implementation. The virtual machine could dedicate processor cores exclusively to robot control tasks, install

specialized robotics software packages, and maintain the precise timing requirements for smooth robot operation.

The trade-off involved increased system resource usage compared to WSL2, as the virtual machine required dedicated RAM and processor allocation. However, the AMD Ryzen 3950X's substantial computational resources easily accommodated both the Windows host environment and the Linux virtual machine, making this overhead acceptable.

The virtual machine approach also provided complete isolation between the Windows design applications and the Linux robotics environment, eliminating potential conflicts while preserving access to both software ecosystems within the same physical computer.

3. Robot-PC Communication

Network Configuration and IP Addresses

Think of an IP address as a postal address for devices on a network—it tells other devices exactly where to send information. Just as your home has a unique street address that allows mail delivery, every device on a network needs a unique IP address to receive data. An IP address consists of four numbers separated by dots (like 192.168.1.102), with each number ranging from 0 to 255.

In this project, the PC uses IP address 192.168.1.101 while the robot's control box uses 192.168.1.102. These addresses exist on the same "subnet" (indicated by the shared 192.168.1 prefix), meaning they can communicate directly without requiring internet access or complex routing. This local network approach ensures fast, reliable communication while keeping the robot isolated from external internet threats.

Static IP assignment proves crucial for robotics applications. Unlike dynamic IP addresses that change automatically (common for home internet devices), static addresses remain constant. This consistency allows the robot control software to always know exactly where to find the robot, eliminating connection failures that could occur if addresses changed mid-operation.

The specific address ranges (192.168.1.x) belong to "private" IP address space, reserved for local networks and unavailable on the public internet. This provides an additional security layer, ensuring robot control commands cannot accidentally be sent over the internet or intercepted by external devices.

Understanding Ports and Virtual Machine Networking

If an IP address is like a building's street address, then network ports are like individual apartment numbers within that building. While the IP address tells data where to go, the port number specifies exactly which application or service should receive that data. Ports are numbered from 1 to 65535, with different ranges reserved for different types of services.

For example, web browsers typically use port 80 for regular websites and port 443 for secure websites. Email services use ports 25, 110, or 993 depending on the specific protocol. Robot communication protocols like RTDE use specific port numbers (typically 30004 for RTDE) that both the robot and PC software must agree upon.

When the robot sends data to the PC, it addresses that data to a specific IP address and port combination (like 192.168.1.101:30004). The PC's operating system then routes that incoming data to whichever application is "listening" on that particular port number.

Virtual Machine Networking Complexity

The virtual machine setup introduces an additional networking layer that complicates direct communication. The Windows host PC connects directly to the robot via Ethernet, but the Linux virtual machine runs "inside" the Windows system, creating a nested network environment.

From the robot's perspective, it can only see and communicate with the Windows host PC at 192.168.1.101. The robot has no awareness that a Linux virtual machine exists within that PC. However, the robotics software (ROS, MoveIt, etc.) runs exclusively within the Linux virtual machine, not on the Windows host.

This creates a communication gap: the robot sends data to the Windows PC, but the software that needs to receive and process that data runs in an isolated virtual environment that the robot cannot directly reach.

Port Forwarding Solution

Port forwarding bridges this communication gap by instructing the Windows host to automatically relay specific network traffic to the virtual machine. When properly configured, port forwarding makes the virtual machine appear as if it's directly connected to the robot, despite running within another operating system.

VirtualBox's port forwarding feature works like a mail forwarding service. When robot data arrives at the Windows PC on specific ports (like 30004 for RTDE communication), VirtualBox automatically redirects that data to corresponding ports within the Linux virtual machine. Similarly, when the virtual machine sends commands back to the robot, VirtualBox forwards those commands through the Windows network interface.

Without port forwarding, robot data would arrive at the Windows PC but have nowhere to go, as Windows doesn't run the robotics software needed to interpret and respond to that data. The virtual machine would simultaneously be running robotics software but unable to receive the robot data it needs to function.

Configuring Port Forwarding

Setting up port forwarding requires identifying which specific ports the robotics software uses and creating forwarding rules for each one. RTDE communication typically uses port 30004, while ROS communication may use various ports depending on the specific nodes and topics involved.

In VirtualBox, port forwarding rules specify that traffic arriving at the Windows host on port X should be redirected to the virtual machine on port Y. Often, these port numbers are identical (forwarding port 30004 on the host to port 30004 in the VM), but they can differ if port conflicts exist.

The forwarding configuration must be bidirectional—allowing both incoming robot data to reach the virtual machine and outgoing commands from the virtual machine to reach the robot. This creates a transparent communication tunnel that allows the robot and virtual machine software to communicate as if they were directly connected.

This port forwarding setup is essential for the system's operation, as it enables the Linux-based robotics software to communicate with the robot while preserving the Windows environment needed for design applications. Without proper port forwarding, the robot would remain isolated from the control software, preventing any automated drawing operations.

Understanding Robot Data Communication

The fundamental insight in robot control is that a robotic arm's position can be completely described by six joint angles—one for each of the UR5's rotating joints. Unlike human intuition, which thinks in terms of "move the hand to this location," robot control systems work by

specifying precise angles for each joint. The robot's mechanical design ensures that specific joint angle combinations always result in the same end-effector (hand) position.

This joint-angle approach simplifies communication dramatically. Instead of transmitting complex 3D coordinates, orientations, and movement paths, the computer need only send six angle values to completely specify where the robot should move. Each angle, measured in radians (a mathematical unit for angles), requires only a few bytes of data to transmit with sufficient precision.

The robot continuously broadcasts its current joint angles back to the computer, creating a feedback loop. This real-time position information allows the computer to verify that commanded movements execute correctly and detect any unexpected obstacles or mechanical issues. The simplicity of this data exchange—essentially six numbers flowing each direction—enables the high-frequency communication necessary for smooth robot motion.

Robot Control Box Communication Protocol

The UR5's CB 3.0 control box serves as an intermediary between the computer and the robot's physical joints. The control box contains the robot's primary computer, safety systems, and motor controllers, handling the complex task of translating high-level movement commands into precise motor control signals.

Communication occurs through standard Ethernet protocols, with the control box acting as a specialized computer that "speaks" both network communication (to the PC) and motor control (to the robot joints). This design isolates the PC from the complexity of individual motor control while providing a standardized interface for sending movement commands.

Robot State Monitoring

Beyond simple command transmission, the control box continuously broadcasts comprehensive information about the robot's current operational state. This includes not only the current joint angles, but also joint velocities (how fast each joint is moving), motor currents (indicating load and potential obstacles), and temperature readings from each joint's motor.

The robot operates in distinct operational modes that the control box communicates to the PC. These modes include "Manual" (where human operators can move the robot by hand), "Automatic" (where the robot follows programmed instructions), and "Protective Stop" (where the robot has detected a safety issue and ceased motion). Understanding the current mode

prevents the PC from attempting to send movement commands when the robot cannot or should not respond to them.

Joint limit monitoring represents another critical state communication. Each of the UR5's six joints has physical limits—maximum and minimum angles beyond which the joint cannot rotate without damaging the mechanism. The control box continuously monitors these limits and reports when joints approach their boundaries, allowing the PC to modify planned movements before damage occurs.

Error States and Safety Communication

The control box implements multiple layers of safety monitoring that generate specific error states when triggered. Collision detection systems monitor unexpected resistance to movement, indicating potential contact with obstacles or humans. When detected, the control box immediately enters a "Protective Stop" state and communicates this condition to the PC.

Emergency stop conditions can originate from multiple sources: physical emergency stop buttons, software-detected safety violations, or communication timeouts. When any emergency stop triggers, the control box immediately halts all robot motion and broadcasts the emergency state to the PC. This communication includes specific error codes that help diagnose the cause—whether human-initiated, software-detected, or hardware-triggered.

Temperature monitoring protects the robot's motors from overheating during intensive operations. Each joint's motor includes temperature sensors that the control box monitors continuously. If temperatures exceed safe operating ranges, the control box reduces motor power or initiates protective stops, communicating these thermal states to the PC for appropriate response.

Communication timeout errors occur when the control box loses contact with the PC for extended periods. Rather than continuing with the last received command indefinitely, the robot enters a safe state and waits for communication restoration. This prevents runaway motion if network connections fail during operation.

Feedback Loop Integration

This comprehensive state information creates a feedback loop between the PC and robot that enables sophisticated control strategies. The PC can adjust its commands based on real-time robot state information, slowing movements when approaching joint limits or modifying paths when unexpected loads are detected.

The continuous state monitoring also enables predictive maintenance, as gradual changes in motor currents or temperatures can indicate developing mechanical issues before they cause failures. This information allows maintenance teams to address problems during scheduled downtime rather than during critical operations.

The control box's safety systems operate independently of PC commands, ensuring robot safety even if software crashes or communication fails. This distributed safety approach provides multiple layers of protection while maintaining the flexibility needed for complex robotic operations like precision drawing tasks.

RTDE (Real-Time Data Exchange)

Real-Time Data Exchange (RTDE) represents Universal Robots' protocol for high-frequency, low-latency communication with their robot control boxes. Unlike standard network protocols optimized for large file transfers or web browsing, RTDE prioritizes consistent, predictable timing over maximum data throughput.

RTDE operates at frequencies up to 500Hz, meaning position updates and commands can be exchanged 500 times per second. This high frequency enables smooth robot motion, as each movement gets broken into tiny increments that appear continuous to human observers. Lower communication frequencies would result in visible stuttering or jerky motion as the robot jumps between discrete positions.

The protocol also provides guaranteed timing bounds, ensuring that communication delays remain within acceptable limits for real-time control. This predictability allows motion planning software to accurately coordinate robot movements with other system components, such as synchronized camera captures or coordination with other robots.

Network Debugging and Troubleshooting

Establishing reliable robot communication often requires systematic debugging of network connectivity. Understanding these diagnostic tools proves essential when connection issues arise, as robot systems provide limited feedback about communication failures.

Firewall Configuration

Computer firewalls act as digital security guards, blocking unauthorized network traffic from reaching your computer. While essential for internet security, firewalls can inadvertently block legitimate robot communication, treating the robot's data packets as potential threats.

Windows Firewall, enabled by default, often blocks the specific network ports that ROS and RTDE use for robot communication. Disabling the firewall (or creating specific exceptions for robotics software) removes this barrier. Since the robot operates on an isolated local network without internet access, disabling the firewall poses minimal security risk while eliminating a common source of connection failures.

The firewall's default behavior of blocking unknown applications makes sense for internet-connected software but proves problematic for specialized robotics applications that don't appear in Windows' pre-approved software lists.

Using Ping for Connection Testing

The "ping" command serves as the most basic test of network connectivity between two devices. When you ping an IP address, your computer sends a small test message to that address and measures how long the response takes to return. Think of it as shouting "hello" and timing how long before you hear an echo back.

To ping the robot, open a command prompt (by typing "cmd" in Windows search) and enter: `ping 192.168.1.102`

Successful ping results show response times (typically 1-5 milliseconds for local connections) and confirm that basic network communication works. Failed pings indicate fundamental connectivity issues that must be resolved before attempting robot control.

Verifying Robot IP Address

Confirming that you're actually communicating with the robot (rather than some other device) requires checking the robot's control box display. The UR5's touch screen interface shows network settings, including its current IP address. This physical verification ensures that the address you're pinging actually belongs to the robot rather than an empty network address or different device.

Additionally, successful ping responses should show consistently low response times (under 10 milliseconds). Higher response times or variable delays suggest network configuration issues that could impact real-time robot control, even if basic connectivity exists.

Network troubleshooting follows a logical progression: first verify physical cable connections, then test basic ping connectivity, confirm IP address settings, verify port forwarding configuration, and finally attempt robot-specific communication protocols. This systematic

approach isolates problems to specific network layers, making resolution more straightforward than attempting to debug everything simultaneously.

4. ROS Foundation

What is ROS and Why It Matters

The Robot Operating System (ROS) is not actually an operating system like Windows or Linux. Instead, ROS functions as a communication framework that enables different software programs to work together in controlling robotic systems. Think of ROS as a universal translator that allows software components written by different people, in different programming languages, to communicate seamlessly.

Before ROS, robotics developers faced a fundamental problem: every robot project required building communication systems from scratch. A camera program needed custom code to send data to a vision processing program, which needed different custom code to send results to a motion planning program. This meant useful robotics software remained isolated within individual projects, making code reuse nearly impossible.

ROS solves this by providing standardized communication methods that work across different hardware platforms and programming languages. A camera driver written for one robot can easily be reused on a completely different robot, as long as both systems run ROS. This modularity accelerates development by allowing engineers to focus on their specific expertise rather than rebuilding basic communication infrastructure.

Core ROS Concepts

Nodes

A "node" represents a single software program with a specific responsibility. Each node operates independently, like individual workers in a factory specialized for particular tasks. One node might handle camera operations, another processes images, while a third plans robot movements.

This design provides significant advantages. If one node crashes, the rest continues operating. Individual nodes can be developed and tested in isolation. Different teams can work on separate nodes simultaneously without interference.

In the drawing robot project, separate nodes handle image processing, robot communication, motion planning, and system coordination. Each can be started, stopped, or modified independently.

Topics

Topics serve as named communication channels where nodes share information. Imagine topics as radio frequencies—one node broadcasts information on a specific topic (like "robot_position"), while other nodes tune in to receive that information.

The key advantage is anonymity. A node publishing camera images doesn't need to know which nodes use those images—it simply broadcasts on the "camera_feed" topic. This loose coupling allows systems to expand without modifying existing code.

Topics carry specific data types, ensuring all nodes agree on information format. The robot joint positions topic carries precisely six angle values, while camera topics carry image data with specific dimensions.

Services

While topics handle continuous data streams, services provide request-response communication for specific actions. Think of services like ordering at a restaurant—you make a request and receive a specific response.

In the drawing robot system, services handle discrete actions like "start drawing" or "emergency stop." When the user requests drawing initiation, the system calls the "start_drawing" service, waits for confirmation, then proceeds. This ensures critical operations complete successfully before continuing.

Services also provide error handling. If a request fails, the calling node receives specific error information rather than waiting indefinitely.

ROS as Hardware-Software Abstraction Layer

ROS creates an abstraction layer that separates application logic from hardware-specific details. The drawing software doesn't need to understand UR5 communication protocols or motor control—it simply publishes desired joint angles and trusts another node to handle robot movement.

This abstraction provides flexibility. The drawing software could control a different robot by replacing only the robot driver node. It also enables simulation—a simulated robot node can replace the real driver for testing without hardware.

Launch Files

Launch files serve as automated startup scripts that coordinate multiple ROS nodes with proper configurations. Rather than manually starting each node individually, launch files define which nodes to start, what parameters each should use, and how they interconnect.

Launch File Structure and Benefits

Launch files are written in Python and contain instructions for system startup. They specify which nodes to launch and critical parameters like robot IP addresses, communication frequencies, and safety limits.

The drawing robot's launch file starts the UR5 driver with correct IP configuration, launches MoveIt motion planning with the robot model, starts the drawing control node with image processing parameters, and initializes visualization. Without the launch file, an operator would manually start each component with dozens of parameters—error-prone and difficult to replicate.

Launch files enable conditional logic. The system can detect simulation versus real hardware mode and automatically start appropriate configurations.

5. Image Processing Pipeline

Adaptive Processing Overview

The image processing system employs adaptive parameter selection rather than fixed algorithm settings. Before applying any edge detection, the system analyzes each input image to determine optimal processing parameters. This pre-analysis calculates metrics like edge density, intensity variance, and noise levels to classify images as "low," "medium," or "high" complexity.

This adaptive approach addresses a fundamental challenge: university logos, architectural drawings, and hand-sketched images require dramatically different processing strategies. A clean, high-contrast logo needs minimal noise reduction but aggressive edge thinning, while a

sketchy architectural drawing requires heavy noise filtering but gentler edge detection to preserve subtle lines.

The system implements three parameter preset configurations corresponding to each complexity level. Rather than requiring manual parameter tuning for each image, the adaptive system automatically selects appropriate weights for multi-scale edge detection, threshold values for line thinning, and smoothing factors for path generation

Algorithm Selection and Coordinate Extraction

The processing pipeline prioritizes extracting drawable line sequences rather than preserving image fidelity. Edge detection uses multi-scale Scharr operators with weighted combination—smaller scales capture fine details while larger scales identify major structural elements. The weights adjust based on the pre-analysis, emphasizing fine details for clean images and structural elements for noisy inputs.

Zhang-Suen thinning reduces thick edge regions to single-pixel paths, essential for robotic drawing where the pen tip has fixed width. Without thinning, the robot would attempt to trace every pixel within thick edge regions, creating overlapping scribbles rather than clean lines.

Path optimization converts pixel-based edge data into smooth coordinate sequences suitable for robotic motion. Douglas-Peucker simplification removes unnecessary waypoints while preserving essential geometric features. The system then applies spline smoothing to eliminate pixel-level jaggedness that would cause jerky robot motion.

From Image to Drawing Coordinates

The final processing stage transforms image pixel coordinates into a standardized 800x600 coordinate system regardless of input image dimensions. This normalization ensures consistent behavior across different source images while providing a predictable coordinate space for robot calibration.

The system validates all extracted coordinates against image boundaries and filters sequences below minimum length thresholds. Short sequences often represent noise artifacts rather than intentional drawing elements. The output JSON format contains arrays of coordinate sequences, where each sequence represents a continuous pen stroke from pen-down to pen-up.

6. Robot Control Stack

UR5 ROS2 Driver Integration

The UR5 ROS2 driver establishes TCP communication with the robot's control box through the External Control URCap. This URCap runs as a program on the robot's internal computer, creating a bridge between the robot's proprietary control system and standard network protocols.

The driver operates in "headless mode," meaning it bypasses the robot's touch screen interface for programmatic control. This mode publishes real-time joint states at high frequency while accepting joint trajectory commands through ROS action interfaces.

Critical to smooth operation, the driver implements position and velocity scaling factors that limit maximum robot speeds regardless of commanded trajectories. These safety limits prevent damage from overly aggressive motion commands while ensuring movements remain within mechanical tolerances.

MoveIt2 Motion Planning

MoveIt2 serves dual purposes in this system: inverse kinematics solving and trajectory validation. For each drawing coordinate, MoveIt2 calculates the joint angles required to position the robot's end-effector at that location. This involves solving complex geometric equations that account for the robot's mechanical constraints and joint limits.

The system configures MoveIt2 with the UR5's URDF (Unified Robot Description Format) model, which precisely defines link lengths, joint limits, and collision boundaries. This model enables MoveIt2 to reject impossible poses and warn about potential collisions before attempting movement.

MoveIt2's planning algorithms generate smooth trajectories between joint configurations, avoiding sudden accelerations that could damage hardware or compromise drawing quality. The planner automatically inserts intermediate waypoints when large joint motions are required, ensuring continuous motion paths.

Joint Trajectory Execution

Joint trajectory execution operates through ROS2 action interfaces that provide feedback and cancellation capabilities. Unlike simple topic publishing, actions allow the drawing system to monitor trajectory progress and detect execution failures.

The system validates each trajectory before execution, checking for joint limit violations, excessive velocities, and malformed coordinate data. Invalid trajectories get automatically corrected by replacing problematic waypoints with safe alternatives derived from current robot state.

Trajectory timing uses configurable velocity and acceleration scaling factors that balance drawing speed against precision requirements. Slower movements improve drawing accuracy but increase total execution time, while faster movements risk overshooting target positions due to mechanical backlash and control system delays.

7. Drawing Node Implementation

System Integration and Coordination

The drawing node orchestrates the entire drawing process through a state machine that coordinates image loading, robot calibration, trajectory planning, and execution monitoring. The node maintains awareness of robot state, safety conditions, and drawing progress throughout operation.

Service interfaces provide external control over drawing operations, allowing users to start drawing, perform emergency stops, or initiate calibration routines. These services implement proper error handling and status reporting, ensuring reliable operation even when subsystems encounter problems.

The node implements comprehensive logging that records drawing progress, execution times, and any encountered errors. This telemetry proves essential for debugging system issues and optimizing performance for different drawing applications.

Image-to-Robot Coordinate Transformation

The coordinate transformation employs bilinear interpolation between four calibrated corner positions. During calibration, operators manually position the robot at the four corners of the drawing area, recording the joint angles for each position. These angles define the mapping between image coordinates and robot joint space.

For any image coordinate, the system normalizes the position within the image bounds (0 to 1 in both dimensions), then interpolates between the corner joint positions using the normalized

coordinates as weights. This approach automatically accounts for the robot's kinematic constraints and workspace limitations.

The transformation includes validation checks that reject coordinates resulting in impossible joint configurations. When invalid coordinates are detected, the system substitutes safe alternative positions rather than attempting impossible movements that could damage equipment.

Trajectory Planning and Execution

The drawing sequence execution alternates between pen-up and pen-down movements. Before drawing each line sequence, the robot moves to the starting position with the pen lifted, lowers the pen to the drawing surface, traces the complete sequence, then lifts the pen again.

Pen lift/lower operations use calibrated joint offsets that vertically translate the robot's position without changing its horizontal location. These offsets are determined during calibration by measuring the difference between touching and lifted positions at the drawing origin.

The system implements trajectory chunking for long drawing sequences, breaking them into smaller segments that can be individually planned and executed. This approach prevents memory issues with very long trajectories while maintaining drawing continuity through careful waypoint management between segments.

Error recovery mechanisms handle common execution failures like joint limit violations or communication timeouts. When errors occur, the system attempts to return the robot to a safe home position before reporting the failure condition to the user.

Final Thoughts

Reality Check

While conceived as beginner-friendly, this project proved considerably complex. Technical challenges like real-time kernel configuration, network debugging, and coordinate transformations require extensive experience to solve effectively.

Essential skills include solid Python and C++ programming. Python handles image processing and coordination; C++ manages robot control and real-time communication. Robotics problems

rarely isolate to single components—debugging requires understanding how all system layers interact.

Design Philosophy

Understand constraints first. Map physical limitations, timing requirements, and safety considerations before selecting technologies. Safety often overrides all other decisions.

Choose robust algorithms over elegant ones. Test with realistic data under actual operating conditions, not idealized benchmarks.

Design with use case in Mind. Consider operator background, usage patterns, and failure scenarios throughout the complete system lifecycle.

Decompose systematically. Break projects into independent subsystems: network communication, robot control, image processing, coordinate transformation, trajectory execution.

Validate incrementally. Start simple and gradually increase complexity. Test failure conditions during development, not after integration.

Embrace iteration. Initial implementations will require significant revision. Plan for fundamental architecture changes based on real-world testing.

Every complex project begins feeling impossible. Break it into digestible pieces, solve each component individually, then connect them. Throughout the course of this project, the problem-solving skills I developed proved far more rewarding than the final system.

I hope this project showcases a glimpse of what is possible with a robot arm and computer–hopefully this write-up provides an ample amount of background knowledge for you to get started creating your own system!

Glossary

This project references various terms that are used ubiquitously throughout computer science and robotics—this section compiles all of the relevant definitions present as well as additional terms that were not explained in their respective sections sorted in alphabetical order.

A

Abstraction Layer: A software interface that hides complex implementation details while providing simplified access to underlying functionality. ROS serves as an abstraction layer between application code and hardware-specific robot drivers.

Action Interface: A ROS2 communication pattern that provides feedback and cancellation capabilities for long-running tasks. Unlike topics (fire-and-forget), actions allow monitoring progress and handling failures during trajectory execution.

Adaptive Processing: An image processing approach that automatically adjusts algorithm parameters based on input image characteristics rather than using fixed settings for all images.

B

Bilinear Interpolation: A mathematical technique for estimating values at unknown points using weighted averages of four known corner values. Used in coordinate transformation to map image pixels to robot joint angles.

C

CB 3.0 Control Box: The third-generation control unit for Universal Robots systems that serves as the robot's "brain," containing computers, safety systems, and communication interfaces.

Cobot (Collaborative Robot): A robotic system designed to work safely alongside humans without requiring safety cages, unlike traditional industrial robots.

Coordinate System: A reference framework for specifying positions in space. The project uses multiple coordinate systems: image pixels, normalized drawing coordinates, and robot joint angles.

D

Degrees of Freedom (DOF): The number of independent motions a robotic system can perform. The UR5 has 6-DOF, allowing it to position and orient its end-effector in three-dimensional space.

Douglas-Peucker Simplification: An algorithm that reduces the number of points in a curve while preserving its essential geometric shape, used to optimize robot drawing paths.

Dynamic IP Address: Network addresses that change automatically over time, as opposed to static addresses that remain constant.

E

Edge Detection: Image processing techniques that identify boundaries between different regions in an image, typically used to extract drawable line elements.

Emergency Stop: Safety mechanisms that immediately halt all robot motion when triggered by hardware buttons, software detection, or communication failures.

End-Effector: The tool or device attached to the end of a robotic arm (in this case, a drawing pen).

Ethernet: A standard networking technology for connecting devices over cables, used for robot-PC communication.

External Control URCap: A Universal Robots software plugin that enables external computers to control the robot programmatically rather than through the built-in interface.

F

Firewall: Security software that monitors and controls network traffic, potentially blocking legitimate robot communication if not properly configured.

H

Headless Mode: Operation without a graphical user interface, allowing programmatic control while bypassing manual touch screen interfaces.

I

Inverse Kinematics: The mathematical process of calculating joint angles required to achieve a desired end-effector position and orientation.

IP Address: A unique numerical identifier assigned to each device on a network, consisting of four numbers separated by dots (e.g., 192.168.1.101).

J

Joint Angles: The rotational positions of each of the robot's six joints, measured in radians, which completely define the robot's pose.

Joint Limits: Physical boundaries that restrict how far each robot joint can rotate, preventing mechanical damage.

JSON (JavaScript Object Notation): A text-based data format used to store and transmit structured information, used for coordinate sequence storage.

K

Kernel: The core component of an operating system that manages hardware resources and provides basic services to applications.

L

Launch File: Python scripts that automate the startup of multiple ROS nodes with proper configurations and parameters.

Linux: An open-source operating system family preferred for robotics applications due to its real-time capabilities and extensive robotics software support.

M

MoveIt2: A ROS2 framework that provides motion planning, inverse kinematics, and trajectory generation for robotic manipulators.

Multi-scale Edge Detection: Image processing technique that analyzes edges at different scales to capture both fine details and major structural elements.

N

Network Port: A numerical identifier that specifies which application or service should receive incoming network data on a device.

Node: In ROS, an independent software program with a specific responsibility that communicates with other nodes through standardized interfaces.

Normalization: The process of converting data to a standard scale or format, such as mapping image coordinates to a 0-1 range regardless of original image size.

P

Payload: The maximum weight a robot can carry, measured in kilograms (the UR5 has a 5kg payload capacity).

Ping: A network utility that tests basic connectivity between devices by sending test messages and measuring response times.

Port Forwarding: Network configuration that redirects traffic from one device to another, enabling communication between robots and software running in virtual machines.

Private IP Address: Network addresses (like 192.168.x.x) reserved for local networks and not accessible from the internet.

Protective Stop: A safety state where the robot immediately ceases all motion due to detected hazards or violations.

R

Real-Time Kernel: An operating system modification that guarantees critical tasks execute within specified time windows, essential for smooth robot control.

Real-Time Data Exchange (RTDE): Universal Robots' high-frequency, low-latency communication protocol for robot control and monitoring.

Robot Operating System (ROS): A framework providing communication tools and libraries for robotics applications (not actually an operating system).

ROS2: The newer version of ROS with improved real-time capabilities, security features, and multi-platform support.

S

Scharr Operator: An edge detection algorithm that identifies boundaries in images by calculating intensity gradients.

Service: A ROS communication pattern for request-response interactions, used for discrete actions like starting drawing or emergency stops.

Spline Smoothing: A mathematical technique for creating smooth curves through a set of points, used to eliminate pixel-level jaggedness in robot paths.

State Machine: A software design pattern that manages system behavior by defining distinct operational states and transitions between them.

Static IP Address: Network addresses that remain constant over time, essential for reliable robot communication.

Subnet: A portion of a network where devices share the same address prefix and can communicate directly.

T

TCP (Transmission Control Protocol): A reliable network communication protocol that ensures data arrives correctly and in order.

Topic: A ROS communication channel where nodes can publish and subscribe to continuous data streams.

Trajectory: A planned path through space that specifies positions, velocities, and timing for robot movement.

U

UR5: A Universal Robots collaborative robotic arm with 5-kilogram payload capacity and six degrees of freedom.

URCap: Universal Robots Capability packages - software plugins that extend robot functionality.

URDF (Unified Robot Description Format): A standard format for describing robot mechanical properties, joint limits, and collision boundaries.

V

Virtual Machine: Software that creates an isolated computer environment within another operating system, used to run Linux robotics software on Windows systems.

VirtualBox: Software for creating and managing virtual machines.

W

Windows Subsystem for Linux (WSL2): Microsoft's technology for running Linux environments within Windows, though insufficient for real-time robotics applications.

Z

Zhang-Suen Thinning: An algorithm that reduces thick edge regions in images to single-pixel-wide lines, essential for converting images to drawable paths.

For additional robotics and computer science terminology, consult specialized dictionaries and documentation for ROS2 Humble, Ubuntu 22.04 Jammy Jellyfish, computer vision, and industrial robotics.